

Week 4 Gate 4 – Consumer Driven Contract Testing & UI Validation

Name: Chandana B

Technology Stack

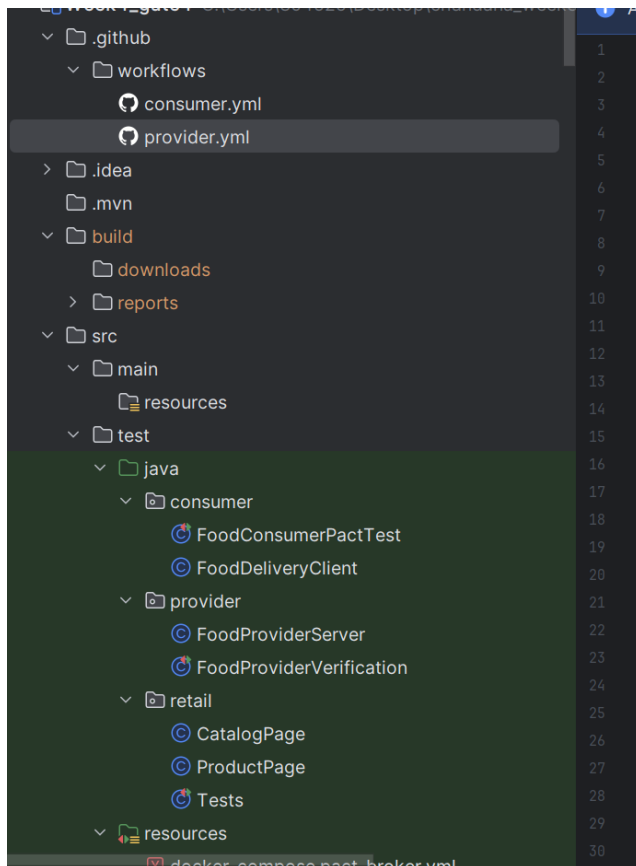
- Java 21
- Maven
- JUnit 5
- REST Assured
- Pact JVM
- PactFlow
- WireMock
- GitHub Actions
- Selenide
- Docker
- Git

Objective

Implement Consumer Driven Contract Testing by separating the Consumer and Provider into independent pipelines. The API contract is shared through PactFlow while UI validation is performed independently using Selenide.

The objective is to ensure that API changes are detected during CI and UI regressions are detected locally.

Project Structure

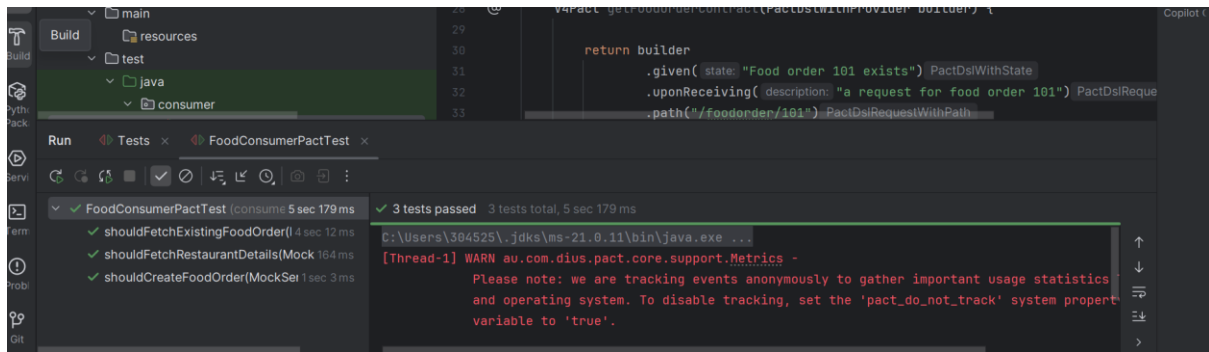


Consumer Pipeline

The consumer pipeline performs the following activities:

1. Checkout source code.
2. Configure Java.
3. Execute Pact Consumer Tests.
4. Generate Pact Contract.
5. Publish Contract to PactFlow.
6. Execute can-i-deploy before deployment.

Consumer produces the contract only. It never communicates directly with the provider.

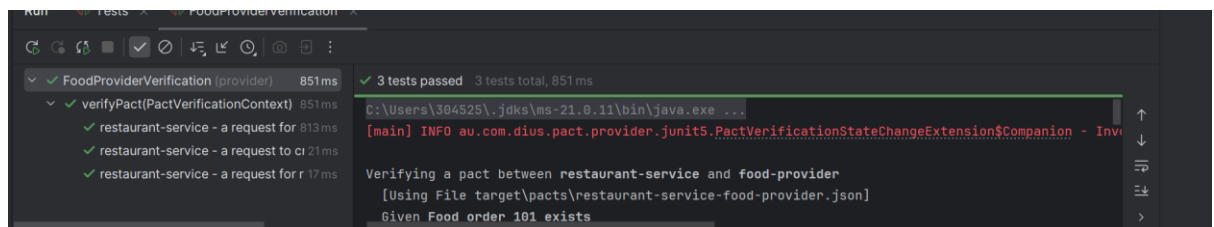


Provider Pipeline

The provider pipeline is completely independent.

Pipeline execution:

1. Fetch latest contract from PactFlow.
2. Start WireMock Provider.
3. Verify every interaction.
4. Publish verification results.
5. Execute can-i-deploy.
6. Deploy only if verification succeeds.



Pact Generated:

```
{
  "consumer": {
    "name": "restaurant-service"
  },
  "interactions": [
    {
```

```
"comments": {
  "testname":
"consumer.FoodConsumerPactTest.shouldFetchExistingFoodOrder(MockServer)",
  "text": [

  ]
},
"description": "a request for food order 101",
"key": "a471e469",
"pending": false,
"providerStates": [
{
  "name": "Food order 101 exists"
}
],
"request": {
  "method": "GET",
  "path": "/foodorder/101"
},
"response": {
  "body": {
    "content": {
      "foodItem": "Veg Pizza",
      "orderId": 101,
      "orderStatus": "CONFIRMED",
      "quantity": 2,
      "restaurantName": "Pizza Hub",
      "statusCode": 200,
```

```
    "totalAmount": 598.0
  },
  "contentType": "application/json",
  "encoded": false
},
"headers": {
  "Content-Type": [
    "application/json"
  ]
},
"matchingRules": {
  "body": {
    "$.foodItem": {
      "combine": "AND",
      "matchers": [
        {
          "match": "type"
        }
      ]
    },
    "$.orderId": {
      "combine": "AND",
      "matchers": [
        {
          "match": "integer"
        }
      ]
    }
  },
}
```

```
"$.orderStatus": {  
  "combine": "AND",  
  "matchers": [  
    {  
      "match": "type"  
    }  
  ]  
},  
"$.quantity": {  
  "combine": "AND",  
  "matchers": [  
    {  
      "match": "integer"  
    }  
  ]  
},  
"$.restaurantName": {  
  "combine": "AND",  
  "matchers": [  
    {  
      "match": "type"  
    }  
  ]  
},  
"$.statusCode": {  
  "combine": "AND",  
  "matchers": [  
    {
```

```
        "match": "integer"
      }
    ]
  },
  "$.totalAmount": {
    "combine": "AND",
    "matchers": [
      {
        "match": "number"
      }
    ]
  }
},
"header": {
  "Content-Type": {
    "combine": "AND",
    "matchers": [
      {
        "match": "regex",
        "regex": "application/json(;.*)?"
      }
    ]
  }
}
},
"status": 200
},
"transport": "http",
```

```
"type": "Synchronous/HTTP"
},
{
  "comments": {
    "testname":
"consumer.FoodConsumerPactTest.shouldCreateFoodOrder(MockServer)",
    "text": [

    ]
  },
  "description": "a request to create a food order",
  "key": "8c52840c",
  "pending": false,
  "providerStates": [
    {
      "name": "Provider can create food orders"
    }
  ],
  "request": {
    "body": {
      "content": {
        "foodItem": "Veg Pizza",
        "orderId": 101,
        "orderStatus": "NEW",
        "quantity": 2,
        "restaurantName": "Pizza Hub",
        "statusCode": 0,
        "totalAmount": 598.0
      }
    }
  }
}
```

```
    },
    "contentType": "application/json",
    "encoded": false
  },
  "headers": {
    "Content-Type": [
      "application/json"
    ]
  },
  "matchingRules": {
    "body": {
      "$.foodItem": {
        "combine": "AND",
        "matchers": [
          {
            "match": "type"
          }
        ]
      },
      "$.orderId": {
        "combine": "AND",
        "matchers": [
          {
            "match": "integer"
          }
        ]
      },
      "$.orderStatus": {
```

```
"combine": "AND",  
"matchers": [  
  {  
    "match": "type"  
  }  
]  
,  
"$quantity": {  
  "combine": "AND",  
  "matchers": [  
    {  
      "match": "integer"  
    }  
  ]  
},  
"$restaurantName": {  
  "combine": "AND",  
  "matchers": [  
    {  
      "match": "type"  
    }  
  ]  
},  
"$statusCode": {  
  "combine": "AND",  
  "matchers": [  
    {  
      "match": "integer"
```

```
    }
  ]
},
"$$.totalAmount": {
  "combine": "AND",
  "matchers": [
    {
      "match": "number"
    }
  ]
}
},
"header": {
  "Content-Type": {
    "combine": "AND",
    "matchers": [
      {
        "match": "regex",
        "regex": "application/json(;.*)?"
      }
    ]
  }
}
},
"method": "POST",
"path": "/foodorder"
},
"response": {
```

```
"body": {
  "content": {
    "foodItem": "Veg Pizza",
    "orderId": 101,
    "orderStatus": "CONFIRMED",
    "quantity": 2,
    "restaurantName": "Pizza Hub",
    "statusCode": 201,
    "totalAmount": 598.0
  },
  "contentType": "application/json",
  "encoded": false
},
"headers": {
  "Content-Type": [
    "application/json"
  ]
},
"matchingRules": {
  "body": {
    "$.foodItem": {
      "combine": "AND",
      "matchers": [
        {
          "match": "type"
        }
      ]
    }
  },
}
```

```
"$.orderId": {  
  "combine": "AND",  
  "matchers": [  
    {  
      "match": "integer"  
    }  
  ]  
},  
"$.orderStatus": {  
  "combine": "AND",  
  "matchers": [  
    {  
      "match": "type"  
    }  
  ]  
},  
"$.quantity": {  
  "combine": "AND",  
  "matchers": [  
    {  
      "match": "integer"  
    }  
  ]  
},  
"$.restaurantName": {  
  "combine": "AND",  
  "matchers": [  
    {
```

```
        "match": "type"
      }
    ]
  },
  "$.statusCode": {
    "combine": "AND",
    "matchers": [
      {
        "match": "integer"
      }
    ]
  },
  "$.totalAmount": {
    "combine": "AND",
    "matchers": [
      {
        "match": "number"
      }
    ]
  }
},
"header": {
  "Content-Type": {
    "combine": "AND",
    "matchers": [
      {
        "match": "regex",
        "regex": "application/json(.*)*?"
      }
    ]
  }
}
```

```
        }
      ]
    }
  }
},
"status": 201
},
"transport": "http",
"type": "Synchronous/HTTP"
},
{
  "comments": {
    "testname":
"consumer.FoodConsumerPactTest.shouldFetchRestaurantDetails(MockServer)",
    "text": [

      ]
    },
    "description": "a request for restaurant details",
    "key": "f49977f9",
    "pending": false,
    "providerStates": [
      {
        "name": "Restaurant Pizza Hub is available"
      }
    ],
    "request": {
      "method": "GET",
```

```
"path": "/restaurant/101"
},
"response": {
  "body": {
    "content": {
      "city": "Chennai",
      "open": true,
      "restaurantId": 101,
      "restaurantName": "Pizza Hub"
    },
    "contentType": "application/json",
    "encoded": false
  },
  "headers": {
    "Content-Type": [
      "application/json"
    ]
  },
  "matchingRules": {
    "body": {
      "$.city": {
        "combine": "AND",
        "matchers": [
          {
            "match": "type"
          }
        ]
      }
    }
  },
}
```

```
"$.open": {  
  "combine": "AND",  
  "matchers": [  
    {  
      "match": "type"  
    }  
  ]  
},  
"$.restaurantId": {  
  "combine": "AND",  
  "matchers": [  
    {  
      "match": "integer"  
    }  
  ]  
},  
"$.restaurantName": {  
  "combine": "AND",  
  "matchers": [  
    {  
      "match": "type"  
    }  
  ]  
}  
},  
"header": {  
  "Content-Type": {  
    "combine": "AND",
```

```
    "matchers": [  
      {  
        "match": "regex",  
        "regex": "application/json(;.*)?"  
      }  
    ]  
  }  
}  
},  
"status": 200  
},  
"transport": "http",  
"type": "Synchronous/HTTP"  
}  
],  
"metadata": {  
  "pact-jvm": {  
    "version": "4.6.20"  
  },  
  "pactSpecification": {  
    "version": "4.0"  
  }  
},  
"provider": {  
  "name": "food-provider"  
}  
}
```

Consumer Contracts

The following interactions were implemented.

GET Food Order

GET /foodorder/101

Expected Response

- Status Code : 200
- Existing Food Order

GET Restaurant

GET /restaurant/101

Expected Response

- Restaurant Information
- Restaurant Availability

POST Food Order

POST /foodorder

Expected Response

- Status Code : 201
- Food Order Created

Provider Verification

Provider verification uses WireMock.

Implemented State Handlers

- Food order 101 exists
- Restaurant Pizza Hub is available
- Provider can create food orders

Each provider state prepares WireMock to return the expected response defined in the contract.

WireMock

WireMock simulates the provider service during verification.

Configured Endpoints

- GET /foodorder/101
- GET /restaurant/101
- POST /foodorder

This allows provider verification without requiring the actual backend service.

Selenide UI Validation

UI automation is maintained independently from contract testing.

Implemented checks include:

- Product search
- Availability badge
- Add to Cart
- Cart validation
- Checkout flow

Selenide validates only the user interface and does not participate in contract verification.

CI/CD Flow

Consumer

Consumer Tests → Generate Pact → Publish to PactFlow → can-i-deploy → Provider
→ Fetch Pact → Verify using WireMock → Publish Verification → can-i-deploy

Conclusion

This implementation follows Consumer Driven Contract Testing principles by separating API contracts from UI testing. API compatibility is validated through PactFlow and WireMock, while user interface behaviour is validated independently using Selenide. This separation allows early detection of API drift and UI regressions, improving deployment confidence and reducing integration failures.